HOCHSCHULE MITTWEIDA
University of Applied Sciences

BACHELORARBEIT

Herr Paul Fuchs

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

2026

Fakultät Angewandte Computer- und Biowissenschaften

BACHELORARBEIT

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

Autor:
Paul Fuchs

Seminargruppe:
MI20w2-B

Studiengang:
Medieninformatik und
Interaktives Entertainment

Erstprüfer:
Prof. Philipp Klimant

Zweitprüfer:
Dr. Beier-Grunwald

Einreichung:
Mittweida, den 27.03.2026

Bibliografische Angaben

Fuchs, Paul: Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen. TODO: 5 Seiten, 14 Abbildungen, 1 Anlage mit Quelltexten, Hochschule Mittweida (FH), Fachbereich Angewandte Computer- und Biowissenschaften

Bachelorarbeit 2026

Abstract TODO

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Problemstellung 1

1.2 Forschungsfrage 1

2 Theoretische Grundlagen **3**

2.1 Microservice Architektur 3

2.2 Microservice Orchestrierung mit Kubernetes 3

2.3 Deployment Strategien 3

2.4 Site Reliability Engineering und dessen Konzepte 3

2.5 Monitoring und Observability 3

3 Konzeption und Methodik **5**

3.1 Systemarchitektur 5

3.2 Theoretisches Modell 7

3.3 Parameter zur Analyse 7

3.3.1 Fehlerfreier Zustand des Systems 7

3.3.2 Fehlerszenario A 8

3.3.3 Fehlerszenario B 8

3.3.4 Analyseintervalle 8

4 Evaluation **9**

4.1 Analyse der injizierten Fehlerrate 9

4.2 Analyse der injizierten Latenz 9

4.3 Fehlerbudget Verbrauch je Deployment Strategie 9

5 Diskussion 11

5.1 Interpretation der Ergebnisse 11

6 Fazit und Ausblick 13

6.1 Beantwortung der Forschungsfrage 13

6.2 Handlungsempfehlungen 13

A Eigenständigkeitserklärung 17

Abbildungsverzeichnis

3.1 Systemarchitektur: EKS Cluster Versuchsaufbau 6

Tabellenverzeichnis

Kapitel 1

Einleitung

1.1 Problemstellung

1.2 Forschungsfrage

Wie verringern Canary Release mit Service Level Objective basierten Analysen den Verbrauch des Fehlerbudgets im Vergleich zu Rolling Updates mit Alertmanager?

Kapitel 2

Theoretische Grundlagen

2.1 Microservice Architektur

2.2 Microservice Orchestrierung mit Kubernetes

2.3 Deployment Strategien

2.4 Site Reliability Engineering und dessen Konzepte

2.5 Monitoring und Observability

Kapitel 3

Konzeption und Methodik

Als sinnbildliches Szenario wurde ein Online-Shop gewählt. Die Architektur des Systems besteht aus zwei Microservices, je ein Repräsentant für den Shop und für die Transaktionen. Zwischen den Services besteht technisch keine Abhängigkeit. Im Microservice für die Transaktionen wird ein simulierter Fehler injiziert, während der Shop-Microservice als Referenz dient, um gegebenenfalls infrastrukturelle Fehler erkennbar zu machen, welche nicht auf die injizierten Fehler zurückzuführen sind. Da es sich hierbei um eine nutzerzentrierte Anwendung handelt, werden zwei Fehlerszenarien überprüft. Einerseits die Injektion einer Fehlerrate, in dem der Service einen internen Serverfehler produziert. Andererseits die Simulation verlängerter Antwortzeit. Es wurde jeweils überprüft, wie lange die jeweilige Analyse benötigt, den Fehler zu erkennen und anschließend den stabilen Zustand widerherzustellen. Weiter wurde berechnet, wie hoch der Verbrauch des Fehlerbudgets innerhalb des getesteten Zeitfensters war.

3.1 Systemarchitektur

Das System wurde in einem AWS EKS Cluster mit zwei m7i-flex.large Nodes aufgesetzt. Diese wurden jeweils dediziert für die Ausführung der Monitoring-Software und Lasttests und die Ausführung der zu testenden Services konfiguriert. Dies verhindert, dass die Messungen durch Konkurrenz um Hardware-Ressourcen beeinflusst werden. Abbildung 3.1 veranschaulicht diesen Zustand noch einmal tiefergehend. Hinzuzufügen ist bei der Systemarchitektur, dass Cilium als Service-Mesh konfiguriert wurde. Dies hätte das Monitoring für die Nutzung der Services

von außerhalb des Clusters vereinfacht. Da die Lasttests jedoch über den Cluster-internen DNS aufgelöst wurden, spielt die Implementierung dessen hier keine weitere Rolle. Die zu testende Applikation besteht aus zwei JVM-basierten Mi-

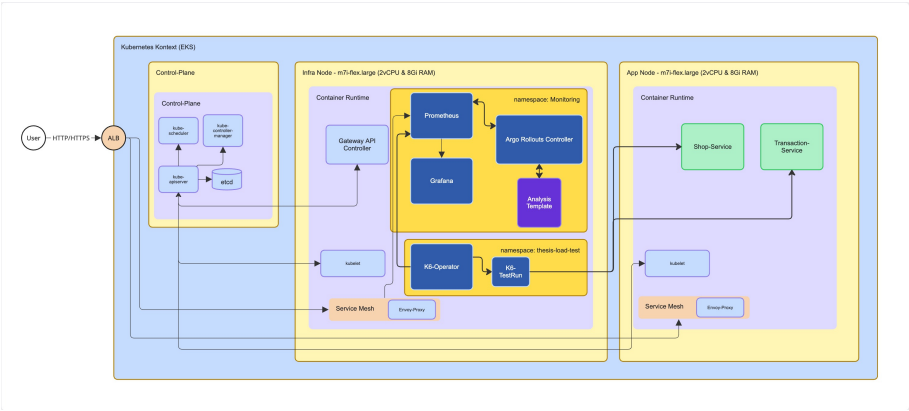

Abbildung 3.1: Systemarchitektur: EKS Cluster Versuchsaufbau

croservices, welche sinnbildlich einen Online-Shop abbilden. Hierfür wurde als Server-Framework Ktor verwendet. Um Fehler während des Deployment Prozesses zu vermeiden, wurden die Container der Microservices jeweils mit Readiness-Probes und einer Mindest-Ready-Zeit von 20 Sekunden ausgestattet. Dies verhindert, dass die Services Anfragen an Pods weiterleten, die sich im Warmup-Prozess befinden. Weiter wurden die Container mit Pre-Stop Hooks von 20 Sekunden versehen, so wird verhindert, dass Anfragen an Pods geleitet werden, die bereits herunterfahren. Um die Erreichbarkeit der Pods sicherzustellen wurden Liveness-Probes implementiert. Diese Applikationen werden auf der Node mit dem Label *role: app-node* gehostet. Die folgenden Software-Tools wurden so konfiguriert, dass sie auf der Node mit dem Label *role: infra-node* ausgeführt werden. Um das Monitoring zu ermöglichen, wurden Prometheus und Grafana im Cluster deployed. Für die Implementierung der jeweiligen Deployment Strategie wurde Argo-Rollouts verwendet. Dies ermöglicht in Verbindung mit Prometheus die SLO-gesteuerte Analyse während des Deployment Vorgangs. Um die Nutzerlast auf den jeweiligen Service zu simulieren wurde K6 mit Constant-Arrival-Rate verwendet. Dies garantiert, dass die Services eine konstant gleichbleibende Last, unabhängig von der Antwortzeit bearbeiten müssen.

3.2 Theoretisches Modell

Rein mathematisch lässt sich bereits beweisen, dass Canary-Releases einen Vorteil beim Verbrauch des Fehlerbudgets gegenüber Rolling Updates haben, in dem Sinne, dass bei gleicher Art des Fehlers, Nutzer kontinuierlich gesteigerten Zugriff auf die neue Softwareversion erhalten. Genauer erklärt am Beispiel: Es wird ein Fehler in der Softwareversion ausgerollt, der in zwei Prozent aller Fälle einen Serverfehler verursacht. Für Rolling Updates erhalten während und nach der Bereitstellung alle Nutzer Zugang zu der neuen fehlerhaften Version, der Einfluss auf den Verbrauch des Fehlerbudgets spiegelt sich direkt wider. Bei Canary Updates werden während des Bereitstellungsprozesses kontinuierlich mehr Nutzer auf die fehlerhafte Version Zugriff erhalten, demnach spiegelt sich während des Deployment Prozesses der Einfluss auf den Verbrauch des Fehlerbudgets anteilig wider. Erst mit dem Abschluss des Canary Releases spiegelt sich der Einfluss direkt auf das Fehlerbudget wider. Weiter ist logisch herzuleiten, dass die Detektionszeit für den Fehler sinkt, wenn sich die Analyse auf die Canary-Version beschränkt, anstatt den Fehler im gesamten System zu überprüfen. Diese Arbeit analysiert den Faktor, um den die Detektionszeit und der Verbrauch des Fehlerbudgets, bei gleicher zeitlicher Analyseintervallrate zwischen Rolling Update und Canary Release mit globaler und isolierter Aggregation sinkt.

3.3 Parameter zur Analyse

Für die Erhebung der Daten wurden jeweils fünf Testläufe über 30 Minuten absolviert. Jeder Service war dabei einer Last von 1200 Anfragen pro Sekunde ausgesetzt. Die injizierten Fehler wurden mittels Umgebungsvariablen realisiert. Da Rolling Updates nativ keine Möglichkeit der Analyse bieten, habe ich mich dazu entschieden, das Alerting von Grafana zu verwenden.

3.3.1 Fehlerfreier Zustand des Systems

Der Zustand der Applikation wurde zuerst ohne injizierte Fehler analysiert, basierend darauf wurde die Definition der SLO's erstellt. Die für diese Arbeit wichtigen Definitionen umfassen:

- eine Verfügbarkeit von 99%
- eine Antwortzeit von 15ms für 95% aller Anfragen

3.3.2 Fehlerszenario A

In Fehlerszenario A wurde der Transaktions-Microservice so konfiguriert, dass er bei der Beantwortung einer Anfrage in 5% der Fälle einen internen Serverfehler meldete. Die zu überprüfende Metrik im Hinblick auf das SLO, ist das Verhältnis fehlerhafter beantworteter Anfragen zur Summe aller Anfragen.

3.3.3 Fehlerszenario B

In Fehlerszenario B wurde der Transaktions-Microservice so konfiguriert, dass er bei der Beantwortung einer Anfrage zusätzlich zur normalen Antwortzeit 15ms warten musste. Die zu überprüfende Metrik ist die Antwortzeit über 95% aller Anfragen, die ohne Fehler beantwortet wurden.

3.3.4 Analyseintervalle

Um ein einheitliches Analyse-Verfahren zu garantieren, wurden die Werte für die Analyseintervalle für alle Analysemethoden vereinheitlicht. Alle 60 Sekunden findet die Überprüfung der Metrik statt. Während für die Analysistemplates ein Fehlerlimit von eins gesetzt wurde, was eine einmalige Überschreitung des Schwellwerts zulässt, wurde der Alertmanager genauso konfiguriert, sodass er bei der ersten Schwellwertüberschreitung den Pending Status annimmt, demnach zulässt und erst bei der zweiten Schwellwertüberschreitung den alarmierten Zustand auslöst.

Kapitel 4

Evaluation

4.1 Analyse der injizierten Fehlerrate

4.2 Analyse der injizierten Latenz

4.3 Fehlerbudget Verbrauch je Deployment Strategie

Kapitel 5

Diskussion

5.1 Interpretation der Ergebnisse

Kapitel 6

Fazit und Ausblick

6.1 Beantwortung der Forschungsfrage

6.2 Handlungsempfehlungen

Literatur

Agrawal, Himanshu (2023). *Kubernetes Fundamentals: A Step-by-Step Development and Interview Guide*. eng. 1st ed. Berkeley, CA: Apress. ISBN: 1484297296.

Argo Rollouts Authors Pittet (2026). *Concepts*. URL: https://argo-rollouts.readthedocs.io/en/stable/concepts/ (besucht am 18. 02. 2026).

Atlassian Authors (2026). *devops*. URL: https://www.atlassian.com/de/devops (besucht am 16. 02. 2026).

AWS Authors (2026). *Microservices*. URL: https://aws.amazon.com/microservices/ (besucht am 20. 01. 2026).

Chawla, Varun (2024). „Exploring How Service Meshes Can Help To Facilitate The Development Of Microservices“. In: *2024 IEEE 15th International Conference on Software Engineering and Service Science (ICSESS)*, S. 35–43. DOI: 10.1109/ICSESS62520.2024.10719389.

Danilo Sato (2014). *Canary Release*. URL: https://martinfowler.com/bliki/CanaryRelease.html (besucht am 06. 02. 2026).

— (5-2014). *Parallel Change*. URL: https://martinfowler.com/bliki/ParallelChange.html (besucht am 20. 02. 2026).

Faseeha, Ummay u. a. (2025). „Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms“. In: *IEEE Access* 13, S. 72011–72039. DOI: 10.1109/ACCESS.2025.3562125.

Forsgren, Nicole, Jez Humble und Gene Kim (2019). *Das Mindset von DevOps. Accelerate*. 1. Aufl. München: Vahlen.

Hasanth, Karumuri Meher und Deepak Nadig (2024). „Evaluating the Performance of Sidecar-based and Sidecarless Cloud-native Service Mesh Solutions“. In: *2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, S. 1–6. DOI: 10.1109/ANTS63515.2024.10897788.

James Lewis, Martin Fowler (2014). *Microservices*. URL: https://www.martinfowler.com/articles/microservices.html (besucht am 17. 01. 2026).

Kubernetes Authors (2026a). *Kubernetes*. URL: https://kubernetes.io/ (besucht am 28. 01. 2026).

— (2026b). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/overview/ (besucht am 22. 02. 2026).

— (2026c). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/overview/ (besucht am 22. 02. 2026).

— (2026d). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/overview/ (besucht am 22. 02. 2026).

— (2026e). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/workloads/ (besucht am 22. 02. 2026).

— (2026f). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/configuration/configmap/ (besucht am 23. 02. 2026).

— (2026g). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/configuration/secrets/ (besucht am 23. 02. 2026).

— (2026h). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/ (besucht am 23. 02. 2026).

— (2026i). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/services-networking/ (besucht am 23. 02. 2026).

Martin Fowler (2015). *Blue Green Deployment*. URL: https://martinfowler.com/bliki/BlueGreenDeployment.html (besucht am 04. 02. 2026).

Pramod Sadalage (5-2016). *Evolutionary Database Design*. URL: https://martinfowler.com/articles/evodb.html (besucht am 18. 02. 2026).

Rakshit, Haranath und Subhasis Banerjee (2024). „Scalability Evaluation on Zero Downtime Deployment in Kubernetes Cluster“. In: *2024 IEEE Calcutta Conference (CALCON)*, S. 1–5. DOI: 10.1109/CALCON63337.2024.10914046.

Sten Pittet (2026). *devops*. URL: https://www.atlassian.com/de/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment (besucht am 16. 02. 2026).

Todd R. Weiss (17-09-2024). *Docker for Web Developers: Getting Started with the Basics*. URL: https://www.docker.com/blog/docker-for-web-developers/ (besucht am 21. 02. 2026).

Anhang A

Eigenständigkeitserklärung

Hiermit erkläre Ich, dass die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Stellen, die wörtlich oder sinngemäß aus Quellen entnommen wurden, sind als solche kenntlich gemacht. Diese Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Borna, den 24.03.2026 Paul Fuchs